

ЧАСТНА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ЗА ДИГИТАЛНИ НАУКИ „СОФТУНИ БУДИТЕЛ“,
гр. София

ДИПЛОМЕН ПРОЕКТ

на Ангел Тончев Делчев

ученик от XII А клас

професия — код: 481030, „Приложен програмист“

специалност — код: 4810301, „Приложно програмиране“

Тема: Уеб базирана платформа за обяви за работа и професионална мрежа „FindAJob“

Ръководител-консултант: Лечо Лечев, Мартин Янев

Сесия: август 2026 г.

Дата: 03/08/2026

СЪДЪРЖАНИЕ

1. [Увод](#)
2. [Изложение](#)
 - 2.1. [Глава 1. Проучвателна част](#)
 - 2.2. [Глава 2. Изисквания към програмния продукт](#)
 - 2.3. [Глава 3. Проектен цикъл на приложението](#)
 - 2.4. [Глава 4. Ръководство за потребителя](#)
3. [Заклучение](#)
4. [Информационни източници](#)
5. [Приложения](#)

1. Увод

Дипломният проект представя разработка в областта на съвременните уеб базирани информационни системи за подбор на персонал и управление на човешки ресурси.

FindAJob е пълнофункционално уеб приложение, проектирано да служи като мост между

работодателите и търсещите работа. В свят, в който дигиталното присъствие е от решаващо значение за кариерното развитие, FindAJob предлага интуитивен интерфейс, съчетан с мощна сървърна логика, за да улесни процеса на подбор на персонал.

Приложението е изградено като **клиент-сървър система**: самостоятелен уеб клиент (Single Page Application) на React и TypeScript, който комуникира със сървърен програмен интерфейс (REST API) на ASP.NET Core чрез HTTP заявки в JSON формат. Данните се съхраняват в релационна база данни SQLite, управлявана чрез Entity Framework Core по подхода Code-First.

Целеви групи на приложението

- **Търсещи работа (Кандидати)** — индивиди, които търсят кариерно развитие. Те имат нужда от платформа, където да могат да изградят детайлен професионален профил, да търсят обяви по ключови думи, да запазват интересни позиции и да кандидатстват максимално бързо, както и да проследяват статуса на всяка своя кандидатура.
- **Работодатели** — компании или HR специалисти, търсещи да привлекат нови таланти. Те разполагат с инструменти за публикуване на детайлни обяви, разглеждане на получените кандидатури, преглед на профила и автобиографията на кандидата и управление на целия процес по подбор.
- **Администратори** — лица, отговарящи за поддръжката и модерацията на платформата. Те контролират потребителските акаунти и ролята им, модерират съдържанието, преглеждат агрегирана статистика и управляват изчакващите регистрации.

Обхват на разработката

Крайният продукт включва:

- **11 контролера** със **76 крайни точки** (endpoints), покриващи автентикация, обяви, кандидатури, профили, автобиографии, съобщения, професионални връзки, известия, запазени обяви, администрация и сървиране на медийни файлове;
 - **23 таблици** в базата данни с **35 индекса**, генерирани чрез **7 миграции**;
 - **29 компонента-страници** в клиентската част, групирани по роля;
 - **80 автоматизирани теста** в два слоя — модулни (unit) и интеграционни (end-to-end).
-

2. Изложение

2.1. Глава 1. Проучвателна част

Преди да започне фазата на проектиране, беше извършено задълбочено изследване на пазара и анализ на утвърдените платформи за търсене и предлагане на работа: **Jobs.bg**, **Indeed** и **LinkedIn**.

Jobs.bg

Най-разпознаваемата и масово използвана платформа на територията на България.

- **Силни страни:** огромна база от потребители и работодатели, силна локализация и изградени навици у местните потребители.
- **Слаби страни:** остарял и статичен потребителски интерфейс. Липсват съвременни социални функции. Комуникацията е тромава, тъй като често се налага преминаване към външни канали (лични имейли или телефонни обаждания), което води до загуба на контекст.

Indeed

Световен агрегатор на обяви за работа.

- **Силни страни:** мощен алгоритъм за търсене, глобален обхват, способност да извлича обяви от корпоративни сайтове.
- **Слаби страни:** претрупан и често объркващ интерфейс. Ниско качество на част от обявите (поради автоматичното им изтегляне), както и слаба директна връзка между кандидата и работодателя.

LinkedIn

Водещата глобална професионална социална мрежа.

- **Силни страни:** богати професионални идентичности, „Easy Apply“ (бързо кандидатстване), отлични възможности за нетуъркинг.
- **Слаби страни:** изключително високи такси за работодателите при публикуване на обяви. Платформата често страда от прекалено много „шум“ под формата на непрофесионално или маркетинг съдържание в информационния поток.

Анализ и изводи

От проведения анализ стана ясно, че традиционните „борси“ (Jobs.bg) са прекалено изолирани и статични, докато социалните мрежи (LinkedIn) са прекалено скъпи и разфокусирани. Липсва хибридно решение — платформа, която да съчетава строгостта и

фокуса на традиционна борса за работа с бързата и интерактивна комуникация на съвременните социални приложения.

Въз основа на горепосочения анализ, в FindAJob бяха внедрени следните решения:

1. **Интегрирана система за съобщения** — премахва нуждата от преминаване към външни имейл платформи. Разговорите се водят вътре в платформата, с индивидуално скриване на разговор от всяка страна и възможност за блокиране на нежелан контакт.
2. **Система за професионални връзки** — позволява изграждането на доверена мрежа от контакти чрез покани, които получателят приема или отхвърля.
3. **Автоматични известия за статус** — кандидатът незабавно разбира, когато неговата кандидатура е прегледана или статусът ѝ е променен, без да се налага да следи таблото си.
4. **Прозрачен процес на подбор** — всяка кандидатура има ясно дефиниран жизнен цикъл (Изчакваща → Прегледана → Интервю → Одобрена/Отхвърлена), видим и за двете страни.

2.2. Глава 2. Изисквания към програмния продукт

Технически изисквания

Изискване	Реализация в проекта
Клиент-сървър архитектура	Разделени проекти: frontend (SPA) и backend (REST API)
Сървърна част	ASP.NET Core 10.0 Web API
Клиентска част	React 19.2 + TypeScript 5.9, компилиран с Vite 8.2
База данни	SQLite чрез Entity Framework Core 10.0, подход Code-First
Първоначално попълване (seeding)	DbInitializer — роли, администратор, демо работодатели и обяви
Минимум 5 обекта (Entities)	Реализирани са 16 собствени обекта + 7 таблици на Identity
Минимум 5 изгледа (Views)	Реализирани са 29 компонента-страници

Изискване	Реализация в проекта
Минимум 5 модела (Models)	Реализирани са 18 файла с модели
Минимум 5 контролера (Controllers)	Реализирани са 11 контролера
Известяване и комуникация	Система за известия + вътрешен чат
Административна част	Панел за потребители, обяви, кандидатури, регистрации и статистика
Потребителски профили	Качване на CV, проследяване на кандидатури, опит, образование, умения
Тестване	80 автоматизирани теста (xUnit) в два слоя
Разгръщане (Deployment)	Docker многоетапен образ, разгърнат във Fly.io

Функционални изисквания по роли

Търсещ работа (Employee):

- регистрация с потвърждение по имейл и възстановяване на забравена парола;
- търсене на обяви по заглавие, компания, локация, описание или таг, с странициране;
- запазване на обява за по-късно и кандидатстване с придружаващо съобщение;
- качване на автобиография във формат PDF, DOC или DOCX;
- изграждане на профил — биография, снимка, банер, професионален опит, образование, умения;
- проследяване на статуса на всяка подадена кандидатура и оттеглянето ѝ;
- изпращане на съобщения и покани за връзка към други потребители.

Работодател (Employer):

- публикуване, редактиране и архивиране на обяви с тагове и структурирани полета;
- преглед на получените кандидатури, на профила и автобиографията на кандидата;
- промяна на статуса на кандидатура през целия жизнен цикъл на подбора;
- поддържане на фирмен профил — размер, индустрия, технологии, придобивки.

Администратор (Admin):

- преглед на агрегирана статистика за платформата;
- търсене и странициране през потребители, обяви и кандидатури;
- редактиране на потребител, промяна на роли, деактивиране и изтриване на акаунт;

- управление на всички обяви, включително архивираните, и присвояване на обява към съответния работодател;
- преглед и премахване на изчакващи регистрации.

Нефункционални изисквания

- **Сигурност** — автентикация чрез бисквитки с флагове **HttpOnly** и **Secure**, ролеви контрол на достъпа, защита срещу подбор на пароли, ограничаване на честотата на заявките.
 - **Производителност** — странициране на всички списъчни отговори, асинхронни операции, индекси върху често търсените колони, режим WAL на базата данни.
 - **Устойчивост на данните** — данните и качените файлове преживяват повторно разгръщане на приложението благодарение на монтиран дисков том.
 - **Използваемост** — адаптивен интерфейс, работещ както на настолен компютър, така и на мобилно устройство.
-

2.3. Глава 3. Проектен цикъл на приложението

2.3.1. Етапи на разработка

Разработката премина през следните етапи:

1. **Анализ на предметната област** — проучване на съществуващите платформи, извеждане на силните и слабите им страни и формулиране на решенията, с които проектът се отличава (Глава 1).
2. **Дефиниране на изискванията** — извеждане на функционалните изисквания по роли и на нефункционалните изисквания за сигурност, производителност и използваемост (Глава 2).
3. **Проектиране на модела на данните** — определяне на обектите, връзките между тях, ограниченията за цялост и индексите. Схемата се създава от C# класовете чрез миграции (Code-First), което позволява всяка промяна да бъде проследена и приложена автоматично.
4. **Реализация на сървърната част** — контролери, услуги, модели на заявките, автентикация и оторизация.
5. **Реализация на клиентската част** — компоненти-страници, контексти за споделено състояние, защита на маршрутите по роля и типизирана комуникация с API-то.
6. **Тестване** — модулни тестове на бизнес логиката и интеграционни тестове на правата за достъп, изпълнявани при всяка промяна.
7. **Оптимизация и укрепване на сигурността** — странициране, индекси, ограничаване на честотата на заявките, защита на качените документи.

8. **Разгръщане** — контейнеризация и публикуване в облачна среда с монтиран дисков том за устойчивост на данните.

Етапите не са строго последователни: тестването и оптимизацията се извършваха успоредно с реализацията, а откритите при тестване проблеми водеха до връщане към по-ранен етап. Този итеративен подход е причината редица дефекти да бъдат намерени и отстранени преди внедряването — например това, че редакцията на обява изтриваше част от полетата ѝ, или че деактивирането на акаунт не прекратяваше вече отворените му сесии.

2.3.2. Избор на технологии

Сървърна част (Backend)

- **ASP.NET Core 10.0 Web API** — осигурява бързодействие, вградена поддръжка за автентикация и оторизация и зрял модел за внедряване на зависимости (Dependency Injection).
- **ASP.NET Core Identity** — управление на потребители, пароли, роли и токени. Използва се само програмният интерфейс (**AddIdentityCore**), без вградените изгледи, тъй като интерфейсът е изцяло на React.
- **Entity Framework Core 10.0** — обектно-реляционно съпоставяне (ORM) по подхода Code-First: схемата на базата се генерира от C# класовете чрез миграции.
- **FluentEmail** — изпращане на транзакционни писма (потвърждение на регистрацията и възстановяване на парола) през SMTP.
- **Bogus** — генериране на реалистични демонстрационни данни при първоначалното попълване.

Клиентска част (Frontend)

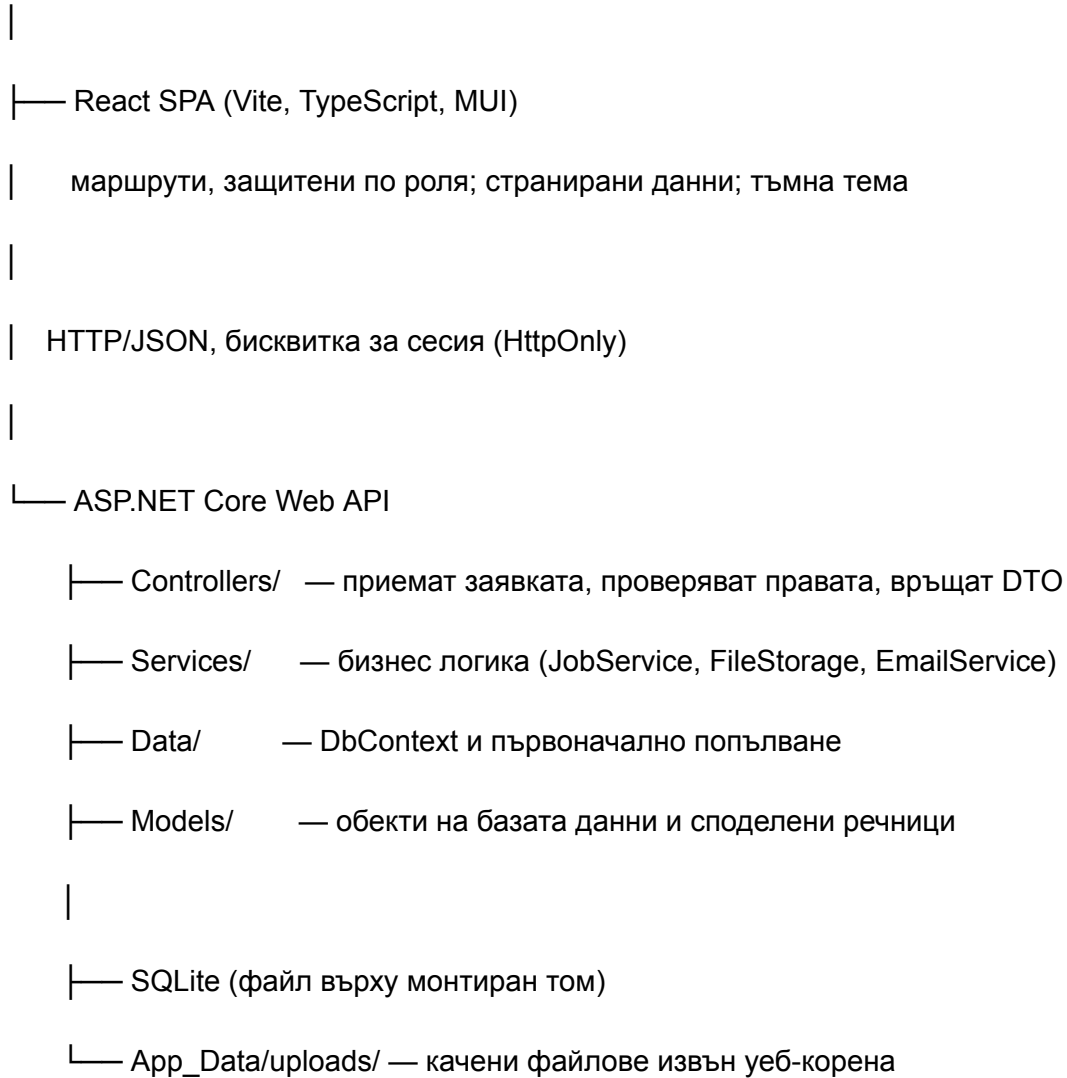
- **React 19.2 с TypeScript 5.9** — компонентен модел с пълна статична типизация на всички отговори от API-то.
- **Vite 8.2** — изключително бърз инструмент за разработка и компилация.
- **Material UI (MUI) 7.3** — библиотека с готови компоненти, конфигурирана с обща тъмна тема. Стилизирането се извършва чрез системата **sx** на MUI и Emotion.
- **React Router 7.18** — клиентска навигация между изгледите.
- **Axios 1.19** — HTTP клиент с включена опция **withCredentials**, за да се предава бисквитката за автентикация.

База данни

- **SQLite** — вграден реляционен двигател, който не изисква отделен сървър. Подходящ за мащаба на проекта и опростява разгръщането: цялата база е един файл върху дисковия том.

2.3.3. Архитектура на приложението

Браузър



Приложението следва **слоеста архитектура**. Контролерите отговарят единствено за приемане на заявката, проверка на правата и връщане на отговор; те не съдържат сложна бизнес логика. Услугите (Service Layer) капсулират логиката, която се използва или която има смисъл да се тества самостоятелно.

В производствена среда **един контейнер обслужва и двете части**: компилираното React приложение се копира в папката **wwwroot** на API-то и се сървира като статично съдържание. Така клиентът и сървърът споделят един произход (origin), което прави бисквитката за автентикация валидна без допълнителни настройки за CORS.

2.3.4. Архитектура на базата данни

Базата е проектирана с висока степен на нормализация. Съдържа **23 таблици**, от които 7 са на ASP.NET Core Identity, а останалите 16 са дефинирани от приложението.

Основни обекти:

Обект	Предназначение
AspNetUsers (ApplicationUser)	Разширява стандартния Identity потребител с FirstName , LastName , CompanyName , ProfessionalTitle
UserProfiles	Разширена информация — биография, адрес, снимка, банер, фирмени данни
JobPostings	Обява за работа: заглавие, описание, изисквания, отговорности, придобивки, локация, заплата, тип заетост, режим на работа, ниво, краен срок, флаг IsDeleted
JobApplications	Подадена кандидатура със статус и връзка към обявата и потребителя
CvDocuments	Метаданни на качените автобиографии — оригинално име, съхранено име, размер, тип
SavedJobs	Запазени от кандидата обяви
Tags / JobPostingTags	Тагове и междинна таблица за връзката „много към много“ с обявите
Messages	Вътрешносистемни съобщения с индивидуално скриване от всяка страна
Notifications	Системни известия — нова кандидатура, промяна на статус, ново съобщение, покана
Friendships / FriendRequests	Професионални връзки и покани за свързване
BlockedUsers	Блокирани контакти

Обект	Предназначение
Experiences / Educations / Skills	Секции от професионалния профил на кандидата
PendingRegistrations	Регистрации, изчакващи потвърждение по имейл

Правила за цялост на данните

Схемата налага правилата на ниво база данни, а не само в кода:

- **Уникални индекси** — една кандидатура на човек за обява (`UserId + JobId`), една запазена обява на човек (`UserId + JobPostingId`), уникално име на таг, един профил на потребител, уникален имейл и токен на изчакваща регистрация.
- **Каскадно изтриване** — изтриването на потребител премахва неговите обяви, кандидатури, профил, автобиографии, известия, опит, образование и умения. Изтриването на обява премахва свързаните с нея кандидатури и тагове.
- **Глобален филтър** — `JobPosting` има филтър `!IsDeleted`, така че архивираните обяви никога не попадат в обикновените заявки. Там, където архивираните са нужни (табло на работодателя, администрация), филтърът се изключва явно с `IgnoreQueryFilters()`.
- **Индекси за производителност** — върху `Title`, `OwnerId` и `CreatedAt` на обявите, върху участниците и времето на съобщенията, върху `UserId` на профилните секции.

Меко изтриване (soft delete)

Обявите не се изтриват физически при архивиране, а се маркират с `IsDeleted`. Това е съзнателно решение: физическото изтриване на обява каскадно унищожава всички подадени по нея кандидатури, което би заличило историята на кандидатите. Архивирането запазва данните и позволява възстановяване.

2.3.5. Разработка на сървърната част

Контролери

Контролер	Отговорност
AuthController	Регистрация, потвърждение по имейл, вход, изход, възстановяване и смяна на парола

Контролер	Отговорност
<code>JobsController</code>	Търсене, преглед, създаване, редактиране и архивиране на обяви
<code>ApplicationController</code>	Подаване на кандидатура, преглед, промяна на статус, оттегляне
<code>ProfilesController</code>	Собствен и публичен профил, качване на снимка и банер, опит, образование, умения
<code>CvController</code>	Качване, изтегляне и изтриване на автобиографии
<code>MessagesController</code>	Пощенска кутия, разговор, изпращане, изтриване, блокиране
<code>FriendshipsController</code>	Покани за връзка, приемане, отхвърляне, премахване
<code>NotificationsController</code>	Списък с известия, брой непрочетени, маркиране като прочетени
<code>SavedJobsController</code>	Запазване и премахване на обяви
<code>AdminController</code>	Потребители, обяви, кандидатури, регистрации, работодатели, статистика
<code>MediaController</code>	Сървиране на публичните изображения от хранилището за файлове

Модел на записа (DTO)

Контролерите **не приемат обекти на базата данни директно** от заявката. За всяка операция по запис е дефиниран отделен модел на заявката с валидационни атрибути. Причината е, че свързването директно към обект на базата позволява на извикващия да зададе полета, които не би трябвало да контролира — например `Id`, `OwnerId`, `IsDeleted` или `CreatedAt` на една обява.

Речниците на приложението (тип заетост, режим на работа, ниво на позицията, статус на кандидатура) са централизирани в класа `JobConstants` и се проверяват на сървъра чрез интерфейса `IValidatableObject`, така че в базата не може да попадне стойност, която интерфейсът не разпознава.

Услуга за обяви (JobService)

Отговаря за извличането, създаването и редактирането на обявите, както и за синхронизацията на таговете през междинната таблица. Търсенето връща **страница от резултати** заедно с общия брой, а не целия набор — при няколкостотин обяви връщането на всички записи с пълните им описания при всяко търсене е неприемливо натоварване.

Пълният код на метода за търсене е даден в **Приложение 1**.

Хранилище за файлове (FileStorage)

Качените файлове се съхраняват **извън веб-корена (wwwroot)**. Това има пряко значение за сигурността: всичко в **wwwroot** се раздава от статичния middleware без каквато и да е проверка за права, което би направило всяка качена автобиография достъпна за всеки, който знае или познае адреса ѝ. При сегашната организация единственият път до файла минава през контролер, който проверява кой пита.

Хранилището генерира ново име на файла (GUID), с което се предотвратява презаписване и се премахва зависимостта от името, подадено от потребителя. Всяко разрешаване на път проверява, че резултатът остава в границите на хранилището — защита срещу атака чрез обхождане на директории (path traversal).

Услуга за електронна поща (EmailService)

Изпраща две писма — за потвърждение на регистрация и за възстановяване на парола — по HTML шаблони, които се копират до изходната директория при компилация. Базовият адрес в препратките идва от конфигурацията, а не е фиксиран в кода, за да сочат писмата от разгърнатия сървър към правилния адрес.

2.3.6. Справочник на приложния програмен интерфейс

Приложението предоставя **76 крайни точки**, разпределени в 11 контролера. Колоната „Достъп“ показва изискваната роля: *публичен* означава достъпен без вход.

Автентикация — **/api/auth**

Метод	Път	Достъп	Предназначение
POST	/register	публичен	Създава изчакваща регистрация и изпраща писмо

Метод	Път	Достъп	Предназначение
GET	/confirm-email	публичен	Потвърждава адреса и създава акаунта
POST	/login	публичен	Вход, връща бисквитка за сесия
POST	/forgot-password	публичен	Изпраща препратка за възстановяване
POST	/reset-password	публичен	Задава нова парола по токен
POST	/change-password	влезли	Смяна на паролата с текущата
POST	/logout	влезли	Изход
GET	/me	влезли	Данни за текущия потребител

Обяви — [/api/jobs](#)

Метод	Път	Достъп	Предназначение
GET	/	публичен	Търсене със странициране
GET	/metadata	публичен	Речници за формата (тип заетост, режим, ниво)
GET	/{id}	публичен	Една обява
POST	/	Admin, Employer	Публикуване
PUT	/{id}	Admin, Employer	Редактиране
DELETE	/{id}	Admin, Employer	Архивиране
GET	/mine	Admin, Employer	Собствени обяви, вкл. архивирани

Метод	Път	Достъп	Предназначение
PUT	/{id}/visibility	Admin, Employer	Архивиране или възстановяване

Кандидатури — [/api/application](#)

Метод	Път	Достъп	Предназначение
POST	/	Employee	Подаване на кандидатура
GET	/mine	Employee	Собствени кандидатури
GET	/employer	Employer	Кандидатури по собствените обяви
PUT	/{id}/status	Admin, Employer	Промяна на статуса
DELETE	/{id}	Admin, Employee	Оттегляне
GET	/{id}/cv	Admin, Employer	Препратка към автобиографията на кандидата

Профили — [/api/profiles](#)

Метод	Път	Достъп	Предназначение
GET / PUT	/me	влезли	Собствен профил
POST	/avatar , /banner	влезли	Качване на изображение
GET	/search	публичен	Търсене на хора със странициране
GET	/{id}	публичен	Публичен профил (без данни за контакт)

Метод	Път	Достъп	Предназначение
POST / DELETE	/experience, /education, /skill	влезли	Секции от профила

Автобиографии — /api/cv

Метод	Път	Достъп	Предназначение
POST	/upload	влезли	Качване (PDF, DOC, DOCX, до 10 MB)
GET	/my	влезли	Списък със собствените документи
GET	/ {id} /content	по правилото за достъп	Изтегляне на документа
DELETE	/ {id}	собственик или Admin	Изтриване

Съобщения, връзки, известия и запазени обяви

Метод	Път	Достъп	Предназначение
GET	/api/messages/inbox	влезли	Разговори с брой непочетени
GET	/api/messages/thread/{id}	влезли	Един разговор; маркира като прочетен
POST	/api/messages	влезли	Изпращане
DELETE	/api/messages/conversation/{id}	влезли	Скриване от собствената кутия
POST / DELETE	/api/messages/block/{id}	влезли	Блокиране и отблокиране

Метод	Път	Достъп	Предназначение
GET	<code>/api/friendships /friends, /requests, /status/{id}</code>	влезли	Връзки и покани
POST	<code>/api/friendships /request/{id}</code>	влезли	Изпращане на покана
POST	<code>/api/friendships /requests/{id}/a ccept, /reject</code>	влезли	Отговор на покана
DELETE	<code>/api/friendships /requests/{id}, /friends/{id}</code>	влезли	Отказ и премахване
GET	<code>/api/notificatio ns/mine, /unread-count</code>	влезли	Известия
POST	<code>/api/notificatio ns/{id}/read, /mark-all-read</code>	влезли	Маркиране като прочетено
GET / POST / DELETE	<code>/api/savedjobs</code>	Employee	Запазени обяви

Администрация — `/api/admin` (всички изискват роля `Admin`)

Метод	Път	Предназначение
GET	<code>/users</code>	Потребители с търсене и странициране
GET	<code>/jobs</code>	Всички обяви, вкл. архивирани, с търсене и странициране
GET	<code>/applications</code>	Всички кандидатури с търсене и странициране

Метод	Път	Предназначение
GET	<code>/employers</code>	Работодатели, на които може да бъде присвоена обява
GET	<code>/stats</code>	Агрегирана статистика
GET / DELETE	<code>/registrations</code>	Изчакващи регистрации
PUT	<code>/users/{id}, /roles, /status</code>	Редакция, роли, деактивиране
DELETE	<code>/users/{id}</code>	Изтриване с всички свързани данни
PUT / DELETE	<code>/jobs/{id}/visibility, /jobs/{id}</code>	Архивиране и окончателно изтриване
PUT / DELETE	<code>/applications/{id}/status, /applications/{id}</code>	Статус и изтриване

Медия — `/uploads/{folder}/{fileName}` — публичен достъп, но само за папките с изображения. Папката с автобиографии е съзнателно изключена.

2.3.7. Пример за поток на заявка: кандидатстване по обява

Следният пример проследява какво се случва при натискане на бутона „Apply now“ и илюстрира разпределението на отговорностите между слоевете.

1. **Клиент.** Компонентът `Apply` изпраща `POST /api/application` с идентификатора на обявата, име, имейл и придружаващо съобщение. `Axios` прикача бисквитката за сесия автоматично (`withCredentials: true`).
2. **Конвейер.** Заявката преминава през ограничителя на честотата, CORS правилото, автентикацията и оторизацията. Атрибутът `[Authorize(Roles = Roles.Employee)]` върху метода отхвърля всеки, който не е кандидат, със статус 403.
3. **Валидация на модела.** Атрибутът `[ApiController]` проверява модела на заявката — задължителни полета, валиден имейл, максимални дължини — и връща 400 с описание на грешките при нарушение.
4. **Идентичност на подателя.** Контролерът **не приема** идентификатора на потребителя от тялото на заявката, а го извлича от удостоверението в бисквитката (`User.FindFirstValue(ClaimTypes.NameIdentifier)`). Това прави невъзможно кандидатстване от чуждо име.

5. **Бизнес проверки.** Проверява се, че обявата съществува и не е архивирана (глобалният филтър я скрива автоматично), че крайният срок не е изтекъл и че този потребител още не е кандидатствал по нея. При повторно кандидатстване се връща 409.
6. **Запис.** Създават се два записа в една транзакция — кандидатурата и известието към работодателя. Уникалният индекс (`UserId`, `JobId`) в базата гарантира правилото „една кандидатура на човек за обява“ дори при едновременни заявки, когато проверката в стъпка 5 не би била достатъчна.
7. **Отговор.** Връща се 200 с идентификатора на кандидатурата. Клиентът показва известие за успех и пренасочва към таблото.
8. **Известяване.** При следващото периодично запитване броячът на непрочетени известия на работодателя се увеличава, а кандидатурата се появява в таблото му.

2.3.8. Сигурност

Сигурността е застъпена на няколко нива:

Автентикация и сесии. Използва се автентикация чрез бисквитка с флаг `HttpOnly` (не е достъпна за JavaScript), `SameSite=Lax` (не се изпраща при заявки от чужд сайт) и `Secure` в производствена среда. Тъй като приложението е API, при липса на права се връща статус код, а не пренасочване към страница за вход.

Ролеви контрол. Три роли — `Admin`, `Employer`, `Employee` — прилагани чрез атрибути [`Authorize(Roles = ...)`]. Освен принадлежността към роля, контролерите проверяват и **собствеността** върху ресурса: работодател може да редактира само собствените си обяви и да управлява само кандидатурите, подадени по тях.

Защита срещу подбор на пароли. Акаунтът се заключва за 15 минути след 5 неуспешни опита. Успоредно с това е конфигуриран ограничител на честотата на заявките: 10 заявки в минута към крайните точки за автентикация от един адрес и допълнителна обща горна граница за целия трафик към тези крайни точки, която не зависи от адреса на клиента и следователно не може да бъде заобиколена чрез подправяне на заглавната част `X-Forwarded-For`.

Прекратяване на сесии. Деактивирането на акаунт и промяната на роли завъртат `SecurityStamp` на потребителя, което обезсилва всички негови активни сесии. Без това деактивираният потребител би запазил пълен достъп до изтичането на бисквитката си.

Достъп до автобиографии. Автобиографията е най-чувствителният документ в системата. Правилото за достъп е: собственикът и администраторът могат винаги; работодател може само докато собственикът има подадена кандидатура по обява на този работодател. Кодът е даден в **Приложение 3**.

Защита на входните данни. Всички низови полета имат ограничение за дължина. Специалните символи % и _ в търсената дума се екранират, преди да влязат в шаблона за `LIKE` — иначе търсене на единичен символ % би съвпаднало с цялата таблица.

Заглавни части на отговора. Задават се `X-Content-Type-Options: nosniff` (браузърът не отгатва типа на съдържанието — важно, защото медийният контролер раздава файлове, качени от потребители), `Referrer-Policy` и `X-Frame-Options: DENY`.

Неразкриване на съществуващи акаунти. Регистрацията и заявката за възстановяване на парола отговарят по абсолютно еднакъв начин независимо дали адресът е зает, за да не могат да бъдат използвани за проверка кой има акаунт в системата.

2.3.9. Процес на регистрация и възстановяване на парола

Регистрация. Данните от формата **не създават потребител веднага**. Те се записват в таблицата `PendingRegistrations` заедно с криптографски случаен токен и срок на валидност от 24 часа, а на посочения адрес се изпраща писмо с препратка за потвърждение. Реалният акаунт се създава едва при отваряне на препратката. Паролата се съхранява хеширана още на този етап и никога не се записва в явен вид. Изтеклите заявки се почистват автоматично.

Възстановяване на парола. Използва се вграденият механизъм за токени на ASP.NET Core Identity, чийто срок на валидност е намален от подразбиращия се един ден на **два часа**. Токенът може да бъде използван само веднъж и е валиден само за акаунта, за който е издаден. Успешната смяна завърта `SecurityStamp`, с което всички останали сесии на акаунта се прекратяват — това е желаното поведение, ако възстановяването се извършва, защото друг човек е знаел старата парола. Смяната също така премахва евентуално наложено заключване.

Смяна на парола от профила. Изисква въвеждане на текущата парола. Прекратява всички останали сесии, но обновява текущата, за да не бъде потребителят изхвърлен от страницата, на която работи.

2.3.10. Разработка на клиентската част

Интерфейсът е изграден с фокус върху „card-based“ дизайн и обща тъмна тема.

Организация на кода

frontend/src/

|— components/ общи компоненти — обвивка на приложението, защита по роля,

	диалози, контроли за списъци, граница за грешки
	└─ providers/ контексти — автентикация, известия, съобщения, потвърждения
	└─ pages/ компоненти-страници, групирани по роля (admin/, employee/, employer/)
	└─ types.ts типове на всички отговори от API-то
	└─ constants.ts споделени списъци с опции и правила за паролата
	└─ jobForm.ts форма за обява — състояние, попълване и преобразуване към заявка
	└─ usePagedList.ts hook за странирани и търсени списъци
	└─ api.ts Axios клиент и нормализация на грешките

Ключови решения

- **Защита на маршрутите.** Компонентът `RequireRole` пренасочва неавтентикирани към страницата за вход и показва екран „Отказан достъп“ на потребител с грешна роля. Сървърът остава истинският арбитър; тази защита само предпазва интерфейса от опити да зареди данни, до които няма да получи достъп.
- **Разделяне на кода (code splitting).** Таблата, регистрацията и съобщенията се зареждат лениво (`React.lazy`), защото са достъпни само след вход. Посетител, който само търси работа, не изтегля техния код.
- **Централизирана обработка на грешките.** Функцията `errorMessage` превежда всяка неуспешна заявка в едно изречение, подходящо за показване — включително мрежови отказ, изтекла сесия, липса на права и грешки от валидацията на сървъра.
- **Граница за грешки (Error Boundary).** Прихваща грешки при изчертаване, за да не остане потребителят пред празна бяла страница.
- **Състояние на търсенето в адреса.** Търсената дума, типът търсене и страницата се пазят в адресната лента, а не в паметта на компонента. Така резултатът може да бъде запазен като отметка, споделен с препратка и достигнат с бутона „назад“ на браузъра.
- **Пълна типизация.** Всички отговори от API-то имат описан тип в `types.ts`.

Основни изгледи

1. **Начална страница** — търсачка с превключване между „Обяви“ и „Хора“, списък с последните публикувани позиции и панел с подробности за избрания резултат.
2. **Детайли за обява** — пълна информация, бързи факти, бутони за кандидатстване и запазване.

3. **Табло на кандидата** — кандидатури, запазени обяви, връзки, покани и профил.
4. **Табло на работодателя** — кандидатури, собствени обяви, връзки, покани и фирмен профил.
5. **Съобщения** — списък с разговори вляво и историята на избрания разговор вдясно.
6. **Административен панел** — статистика и четири раздела с търсене и странициране.

Адаптивност. Под определена ширина на екрана панелът с подробности се скрива; за да не остане резултатът недостъпен на телефон, избирането на обява отваря нейната собствена страница, а избирането на човек — диалог с профила му.

2.3.11. Тестване

Проектът съдържа **80 автоматизирани теста** в отделен проект `backend.tests`, изградени с **xUnit**. Тестовите са разделени на два слоя.

Модулни тестове (unit) — покриват:

- `JobService` — търсене, странициране, правила за собственост, синхронизация на таговете, че редакцията пренася всички полета и че специалните символи в търсенето се третират като текст;
- гаранциите на базата данни — каскадно изтриване, уникални ограничения;
- хранилището за файлове, включително отхвърлянето на опит за обхождане на директории;
- споделените речници за роли и статуси.

Тестовите работят срещу **истинска SQLite база в паметта**, а не срещу доставчика „EF Core In-Memory“. Това е съзнателен избор: доставчикът в паметта не налага външни ключове, каскади и уникални индекси — тоест точно поведението, което тези тестове трябва да проверят.

Интеграционни тестове (end-to-end) — стартират цялото приложение чрез `WebApplicationFactory`, с реалния конвейер, реалните контролери и реалната автентикация с бисквитки. Те покриват това, до което модулните тестове не достигат:

- административните крайни точки отхвърлят анонимни потребители, кандидати и работодатели;
- работодател не може да редактира, архивира или да действа върху чужда обява;
- кандидат не може да публикува обява, нито да променя статуса на собствената си кандидатура;
- автобиография се чете от собственика си и от работодател **само** докато кандидатът има подадена кандидатура по обява на този работодател;

- публичният медиен маршрут раздава снимки, но отказва папката с автобиографии;
- регистрацията не разкрива дали един адрес вече е зает, налага политиката за паролите и не раздава администраторска роля;
- токен за възстановяване на парола работи еднократно, само за акаунта, за който е издаден, и прави старата парола невалидна;
- само администратор може да реши на кой работодател принадлежи една обява.

Пример за интеграционен тест е даден в **Приложение 5**.

2.3.12. Производителност и оптимизация

- **Странициране** — всички списъчни крайни точки връщат страница заедно с общия брой. Административните списъци също се търсят и странират на сървъра, вместо цялата таблица да се изпраща към браузъра.
- **Асинхронно програмиране** — `async/await` при всички входно-изходни операции.
- **Режим WAL** — базата е конфигурирана с Write-Ahead Logging и `synchronous=NORMAL`, което позволява четенето и писането да не се блокират взаимно.
- **Индекси** — 35 индекса върху колоните, по които реално се търси и подрежда.
- **Оптимизирани заявки** — ролите на потребителите се извличат с едно съединение (`join`) вместо с по една заявка на ред; кандидатурите на работодателя се извличат с едно съединение вместо с предварително извличане на идентификаторите на обявите му.
- **Екраниран LIKE** — търсенето използва `LIKE` без извикване на `lower()` за всяка колона на всеки ред. Пълнотекстово търсене (FTS5) беше разгледано и съзнателно отхвърлено: токенизаторът му премахва пунктуацията, което на платформа, чиито тагове са „C#“, „.NET“ и „Node.js“, би превърнало търсенето на „C#“ в търсене на „C“.
- **Разделяне на пакета** — MUI се извежда в отделен файл, за да не обезсилва промяна в приложението кеша на браузъра за библиотеките.

2.3.13. Разгръщане

Проектът е контейнеризиран чрез **многоетапен Dockerfile**:

1. **Етап 1** — компилира React приложението с `npm ci` (инсталира точно версиите от заключващия файл, за да не се получи различно дърво от зависимости от тестваното).
2. **Етап 2** — компилира и публикува API-то.
3. **Етап 3** — изпълнимият образ. Компилираното React приложение се копира в `wwwroot` на API-то, така че един контейнер обслужва един произход.

Разгръщането се извършва във **Fly.io**. Ключова подробност е **монтираният дисков том /data**: върху него живеят и базата данни, и всички качени файлове. Без него всяко ново разгръщане заменя файловата система на контейнера и отнася данните със себе си.

Конфигурацията следва йерархия: **appsettings.json** → файл според средата → променливи на средата. Нищо тайно не се записва в хранилището за код — паролите за първоначалното попълване в производствена среда идват от променливи на средата, а когато липсват, просто не се създава нищо, така че разгърнат сървър никога не остава с известни по подразбиране идентификационни данни.

Платформата следи здравето на приложението чрез крайната точка **GET /health**.

2.4. Глава 4. Ръководство за потребителя

4.1. Регистрация

1. От началния екран се избира бутон **„Register“**.
2. Избира се типът акаунт — търсещ работа или работодател (връзката „Sign up over here“ води към формата за работодател).
3. Попълват се имейл, име, фамилия, парола и адресни данни. Изискванията към паролата се показват в реално време и бутонът остава неактивен, докато всички не са изпълнени.
4. След изпращане на формата на посочения адрес се изпраща писмо с препратка за потвърждение. **Акаунтът се създава едва след отварянето ѝ.**
5. В среда за разработка, когато няма конфигуриран пощенски сървър, на екрана се показва бутон **„Confirm now“**, който изпълнява същото действие.

4.2. Вход и възстановяване на парола

1. Влизането става с имейл или потребителско име и парола.
2. При забравена парола се използва препратката **„Forgot your password?“** под формата. Въвежда се имейл адресът и на него се изпраща препратка за избор на нова парола, валидна два часа.
3. След успешна смяна всички други устройства, на които акаунтът е бил в сесия, се изключват.
4. Паролата може да се смени и от профила чрез бутона **„Change password“**, при което се изисква текущата парола.

4.3. Изграждане на профил (кандидат)

1. От менюто горе вдясно се избира **„My dashboard“**, след което разделът **„My profile“**.

2. Бутонът „**Edit profile**“ отваря формата за лични данни, биография и адрес.
3. Снимката и банерът се сменят чрез иконата с фотоапарат върху тях (JPG, PNG или WEBP, до 5 MB).
4. Секциите „**Experience**“, „**Education**“ и „**Skills**“ се попълват чрез бутона „+“.
5. Автобиография се качва от секцията „**Resumes & CVs**“ (PDF, DOC или DOCX, до 10 MB).
6. Индикаторът „**Profile completeness**“ показва кои части още липсват.

4.4. Търсене и кандидатстване

1. На началната страница се избира типът търсене — „**Jobs**“ или „**People**“ — и се въвежда ключова дума. Търсенето обхваща заглавие, компания, описание, локация и тагове.
2. Резултатите се показват вляво, а подробностите за избрания — вдясно. На малък екран избирането на обява отваря нейната собствена страница.
3. Бутонът „**Save**“ запазва обявата в раздел „**Saved jobs**“ на таблото.
4. Бутонът „**Apply now**“ отваря формата за кандидатстване. Името и имейлът се попълват предварително от профила; добавя се придружаващо съобщение.
5. Автобиографията се прикача автоматично от профила — работодателят вижда основната или последно качената.
6. Не може да се кандидатства два пъти по една и съща обява, нито след изтичане на крайния срок.

4.5. Проследяване на кандидатури (кандидат)

1. Разделът „**Applications**“ на таблото показва всички подадени кандидатури.
2. Всяка кандидатура има статус: **Pending**, **Reviewed**, **Interviewing**, **Accepted** или **Rejected**. При промяна на статуса кандидатът получава известие.
3. Бутонът „**Withdraw**“ оттегля кандидатурата; след това може да се кандидатства отново.

4.6. Управление на обяви (работодател)

1. Бутонът „**Post a job**“ отваря формата за нова обява — заглавие, компания, локация, заплата, тип заетост, режим на работа, ниво, краен срок, тагове, описание, отговорности, изисквания, придобивки и информация за компанията.
2. Разделът „**My jobs**“ показва всички публикувани обяви, включително архивираните.
3. Бутонът „**Edit**“ отваря обявата за редактиране, „**Preview**“ я показва така, както я вижда кандидатът, а „**Archive**“ я скрива от търсенето, без да изтрива получените кандидатури. Архивирана обява може да бъде възстановена.

4.7. Преглед на кандидатури (работодател)

1. Разделът „**Applications**“ показва всички кандидатури по обявите на работодателя.

2. Бутонът „**Profile**“ отваря профила на кандидата, „**View CV**“ — автобиографията му, а „**Message**“ започва разговор с него.
3. Статусът се променя от падащото меню в реда на кандидатурата; кандидатът получава автоматично известие.

4.8. Съобщения и професионални връзки

1. Разговор може да се започне от профила на потребител, от списъка с връзки или от бутона „**Message**“ върху кандидатура.
2. Разделът „**Messages**“ в горната лента показва разговорите вляво и историята вдясно. Изпращането става с бутона „**Send**“ или с клавиша Enter.
3. Изтриването на разговор го премахва само от собствената пощенска кутия; другата страна запазва своето копие.
4. Блокирането на потребител прекратява възможността за размяна на съобщения в двете посоки.
5. Покана за връзка се изпраща от профила на потребителя с бутона „**Connect**“, а получените покани се приемат или отхвърлят от раздела „**Requests**“.

4.9. Известия

Иконата със звънец в горната лента показва броя непочетени известия и се обновява автоматично. Страницата с известия позволява отваряне на съответния екран или маркиране като прочетено, поединично или наведнъж.

4.10. Административен панел

1. Достъпен е само за роля **Admin** през „**Administration**“.
2. Горната част показва обобщена статистика — брой потребители, обяви, кандидатури и разпределение по роли.
3. Разделът „**Jobs**“ позволява търсене, странициране, създаване, редактиране, архивиране и окончателно изтриване на обяви. При създаване или редактиране задължително се избира **работодателят**, на когото принадлежи обявата.
4. Разделът „**Users**“ позволява търсене на потребители, преглед на профил, промяна на данни и роли, деактивиране и изтриване. Администратор не може да премахне собствената си роля, да се деактивира или да изтрие себе си или друг администратор.
5. Разделът „**Applications**“ дава преглед и управление на всички кандидатури.
6. Разделът „**Registrations**“ показва изчакващите потвърждение регистрации.

3. Заключение

Проектът „FindAJob“ успешно демонстрира проектирането, изграждането, тестването и внедряването на сложна уеб базирана информационна система. Спазени са стриктно

всички изисквания на заданието — разделяне на клиентска и сървърна част, релационна база данни с Code-First подход и първоначално попълване, ролеви контрол на достъпа, система за комуникация и известяване, административен модул, автоматизирано тестване и разгръщане в интернет среда чрез контейнеризация.

Освен изпълнението на формалните изисквания, в хода на разработката бяха решени и редица проблеми, характерни за реални производствени системи: защита на чувствителни документи от неоторизиран достъп, устойчивост на данните при повторно разгръщане, предотвратяване на загуба на данни при редакция, ограничаване на честотата на заявките и незабавно прекратяване на сесиите на деактивирани акаунти. Именно тези решения превръщат проекта от учебен пример в работеща система.

Възможности за бъдещо развитие:

- алгоритъм с изкуствен интелект за препоръчване на подходящи обяви спрямо уменията и опита на кандидата;
- модул за провеждане на видео интервюта директно през платформата;
- обмен на съобщения в реално време чрез WebSocket (SignalR) вместо периодично запитване;
- миграция към PostgreSQL с индекси за размито търсене при нарастване на обема данни;
- разширена аналитика за работодатели — брой прегледи на обява и конверсия към кандидатура;
- мобилно приложение, използващо съществуващия REST API.

4. Информационни източници

1. Microsoft Documentation. „ASP.NET Core Documentation“. <https://learn.microsoft.com/en-us/aspnet/core/>
2. Microsoft Documentation. „Entity Framework Core“. <https://learn.microsoft.com/en-us/ef/core/>
3. Microsoft Documentation. „Introduction to Identity on ASP.NET Core“. <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>
4. Microsoft Documentation. „Rate limiting middleware in ASP.NET Core“. <https://learn.microsoft.com/en-us/aspnet/core/performance/rate-limit>
5. Microsoft Documentation. „Integration tests in ASP.NET Core“. <https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests>
6. React Documentation. „React – The library for web and native user interfaces“. <https://react.dev/>
7. TypeScript Documentation. „TypeScript: JavaScript With Syntax For Types“. <https://www.typescriptlang.org/>

8. MUI. „Material UI – React component library“. <https://mui.com/material-ui/>
 9. Vite. „Vite – Next Generation Frontend Tooling“. <https://vite.dev/>
 10. React Router. „React Router Documentation“. <https://reactrouter.com/>
 11. xUnit.net. „xUnit.net documentation“. <https://xunit.net/>
 12. SQLite. „SQLite Documentation“ (Write-Ahead Logging, LIKE optimization).
<https://www.sqlite.org/docs.html>
 13. OWASP. „OWASP Top Ten“. <https://owasp.org/www-project-top-ten/>
 14. Fly.io. „Fly.io Documentation“. <https://fly.io/docs/>
 15. Docker. „Docker Documentation – Multi-stage builds“. <https://docs.docker.com/build/building/multi-stage/>
-

5. Приложения

Приложение 1: Търсене на обяви със странициране (**JobService.cs**)

Методът връща страница резултати заедно с общия брой. Търсената дума се превръща в екраниран шаблон за **LIKE**, за да не могат специалните символи да променят смисъла на заявката.

```
public async Task<PagedResult<JobPosting>> SearchJobsAsync(
    string? searchTerm,
    int page = 1,
    int pageSize = DefaultPageSize,
    CancellationToken cancellationToken = default
)
{
    page = Math.Max(page, 1);
    pageSize = Math.Clamp(pageSize, 1, MaxPageSize);
    await using var context = await _factory.CreateDbContextAsync(cancellationToken);
    var query = context
```

```

        .JobPostings.AsNoTracking()

        .Include(j => j.JobPostingTags)

        .ThenInclude(jt => jt.Tag)

        .AsQueryable();

    if (!string.IsNullOrEmpty(searchTerm))
    {
        var pattern = SearchPattern.Contains(searchTerm);

        query = query.Where(j =>

            EF.Functions.Like(j.Title, pattern, SearchPattern.Escape)

            || EF.Functions.Like(j.Company, pattern, SearchPattern.Escape)

            || EF.Functions.Like(j.Description, pattern, SearchPattern.Escape)

            || EF.Functions.Like(j.Location, pattern, SearchPattern.Escape)

            || j.JobPostingTags.Any(jt =>

                jt.Tag != null && EF.Functions.Like(jt.Tag.Name, pattern, SearchPattern.Escape)

            )

        );
    }

    var total = await query.CountAsync(cancellationToken);

    var jobs = await query

        .OrderByDescending(j => j.CreatedAt)

        .ThenByDescending(j => j.Id)

```

```

        .Skip((page - 1) * pageSize)

        .Take(pageSize)

        .ToListAsync(cancellationToken);

    jobs.ForEach(PopulateTags);

    return new PagedResult<JobPosting>(jobs, page, pageSize, total);
}

```

Приложение 2: Екраниране на шаблона за търсене (SearchPattern.cs)

```

public static class SearchPattern
{
    /// <summary>Знакът за екраниране, който се подава на LIKE ... ESCAPE.</summary>

    public const string Escape = "\\";

    /// <summary>Шаблон „съдържа този текст“ с неутрализирани заместващи
    символи.</summary>

    public static string Contains(string term) => $"%{EscapeWildcards(term.Trim())}%";

    private static string EscapeWildcards(string term) =>

        term.Replace("\\", "\\").Replace("%", "\\%").Replace("_", "\\_");
}

```

Приложение 3: Правило за достъп до автобиография (CvController.cs)

Най-чувствителната проверка в системата. Работодател получава достъп единствено докато собственикът на документа има подадена кандидатура по обява, която принадлежи на този работодател.

```

/// <summary>

```

/// Собственикът и администраторите могат винаги да четат автобиография. Работодател

/// може само докато собственикът има подадена кандидатура по обява на този работодател.

/// </summary>

```
private async Task<bool> CanAccessAsync(CvDocument cv, string userId)
```

```
{  
    if (cv.UserId == userId || User.IsInRole(Roles.Admin))
```

```
{  
        return true;
```

```
}  
    if (!User.IsInRole(Roles.Employer))
```

```
{  
        return false;
```

```
}  
    return await _context.JobApplications.AnyAsync(a =>
```

```
        a.UserId == cv.UserId  
        && _context.JobPostings.IgnoreQueryFilters()  
        .Any(j => j.Id == a.JobId && j.OwnerId == userId)
```

```
);
```

```
}
```

Приложение 4: Правила за цялост на данните (`ApplicationContext.cs`)

Извадка от конфигурацията на модела. Правилата се налагат от базата данни, а не само от кода.

```
protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);

    // Архивираните обяви не участват в обикновените заявки.
    builder.Entity<JobPosting>().HasQueryFilter(j => !j.IsDeleted);

    builder.Entity<JobPosting>().HasIndex(j => j.Title);

    builder.Entity<JobPosting>().HasIndex(j => j.OwnerId);

    builder.Entity<JobPosting>().HasIndex(j => j.CreatedAt);

    builder.Entity<UserProfile>().HasIndex(p => p.UserId).IsUnique();

    builder.Entity<Tag>().HasIndex(t => t.Name).IsUnique();

    builder.Entity<SavedJob>().HasIndex(s => new { s.UserId, s.JobPostingId }).IsUnique();

    // Обслужва пощенската кутия и разговора — филтрира по двамата участници
    // и подрежда по време.

    builder.Entity<Message>().HasIndex(m => new { m.SenderUserId, m.ReceiverUserId,
m.SentAt });

    // Правилото „една кандидатура на човек за обява“ се налага от базата данни,
    // а не само от проверката в контролера.

    builder.Entity<JobApplication>().HasIndex(a => new { a.UserId, a.JobId }).IsUnique();

    // Каскадно изтриване
```

builder

```
.Entity<JobPosting>()  
.HasMany(j => j.JobPostingTags)  
.WithOne(jt => jt.JobPosting)  
.HasForeignKey(jt => jt.JobPostingId)  
.OnDelete(DeleteBehavior.Cascade);
```

builder

```
.Entity<ApplicationUser>()  
.HasMany<JobApplication>()  
.WithOne()  
.HasForeignKey(a => a.UserId)  
.OnDelete(DeleteBehavior.Cascade);
```

}

Приложение 5: Интеграционен тест за достъп до автобиография ([AuthorizationTests.cs](#))

Двата теста са двойка: първият доказва, че достъпът се отказва, а вторият — че при наличие на кандидатура се разрешава. Без втория тест първият би минавал дори ако крайната точка беше счупена за всички.

[Fact]

```
public async Task AnEmployerCannotReadTheCvOfSomebodyWhoNeverAppliedToThem()
```

```
{
```

```
    var (seeker, _) = await SignedInAsync("cv-stranger", Roles.Employee);
```

```
    var (employer, _) = await SignedInAsync("cv-nosy-employer", Roles.Employer);
```



```

    var cvId = await UploadCvAsync(seeker);

    var response = await employer.GetAsync($"api/cv/{cvId}/content");

    Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
}

[Fact]

public async Task AnEmployerCanReadTheCvOfSomebodyWhoAppliedToThem()
{
    var (seeker, _) = await SignedInAsync("cv-applicant", Roles.Employee);

    var (employer, _) = await SignedInAsync("cv-hiring-employer", Roles.Employer);

    var cvId = await UploadCvAsync(seeker);

    var jobId = await CreateJobAsync(employer);

    await SubmitApplicationAsync(seeker, jobId);

    var response = await employer.GetAsync($"api/cv/{cvId}/content");

    Assert.Equal(HttpStatusCode.OK, response.StatusCode);
}

```

Приложение 6: Първоначално попълване на базата (DbInitializer.cs)

Попълването е **добавящо**: нищо не се изтрива и въведено през интерфейса съдържание никога не се презаписва. Демонстрационни обяви се генерират само срещу база, в която няма нито една обява — това прави рестартите безопасни, защото кандидатурите и запазените обяви сочат към тези редове.

```

private static async Task SeedAdminAsync(

    UserManager<ApplicationUser> userManager,

```

```

IConfigurationSection options,

ILogger logger

)

{

    var email = options["AdminEmail"] ?? "monkey@findajob.com";

    var userName = options["AdminUserName"] ?? "monkey";

    var password = options["AdminPassword"];

    if (await userManager.FindByEmailAsync(email) is not null

        || await userManager.FindByNameAsync(userName) is not null)

    {

        return;

    }

    // Без конфигурирана парола не се създава акаунт, за да не остане разгърнат

    // сървър с известни по подразбиране идентификационни данни.

    if (string.IsNullOrEmpty(password))

    {

        logger.LogWarning(

            "No Seed:AdminPassword is configured, so the administrator account was not created. "

            + "Set it with `dotnet user-secrets set Seed:AdminPassword <value>` or the "

            + "SEED__ADMINPASSWORD environment variable."

        );

        return;
    }

```

```

    }

    var admin = new ApplicationUser

    {
        UserName = userName,

        Email = email,

        EmailConfirmed = true,

        CompanyName = "FindAJob Headquarters",

        ProfessionalTitle = "System Administrator",

    };

    var result = await userManager.CreateAsync(admin, password);

    if (result.Succeeded)

    {

        await userManager.AddToRoleAsync(admin, Roles.Admin);

        logger.LogInformation("Seeded administrator account {Email}.", email);

    }

}

```

Приложение 7: Конфигурация на сигурността (Program.cs)

builder

```

.Services.AddIdentityCore<ApplicationUser>(options =>

{

    options.SignIn.RequireConfirmedAccount = true;

```

```
options.User.RequireUniqueEmail = true;

options.Password.RequiredLength = 8;

options.Password.RequireDigit = true;

options.Password.RequireLowercase = true;

options.Password.RequireUppercase = true;

options.Password.RequireNonAlphanumeric = true;

// Повтарящите се неуспешни опити заключват акаунта.

options.Lockout.AllowedForNewUsers = true;

options.Lockout.MaxFailedAccessAttempts = 5;

options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);

})

.AddRoles<IdentityRole>()

.AddEntityFrameworkStores<ApplicationDbContext>()

.AddSignInManager()

.AddDefaultTokenProviders();

// Препратките за възстановяване на парола са краткотрайни.

builder.Services.Configure<DataProtectionTokenProviderOptions>(options =>

{

    options.TokenLifespan = TimeSpan.FromHours(2);

});

// След колко време сесията забелязва, че SecurityStamp е бил завъртян — това правят

// деактивирането на акаунт и промяната на роли.
```

```
builder.Services.Configure<SecurityStampValidatorOptions>(options =>
{
    options.ValidationInterval = TimeSpan.FromMinutes(5);
});

builder.Services.ConfigureApplicationCookie(options =>
{
    options.Cookie.HttpOnly = true;

    options.Cookie.SameSite = SameSiteMode.Lax;

    options.Cookie.SecurePolicy = builder.Environment.IsDevelopment()
        ? CookieSecurePolicy.None
        : CookieSecurePolicy.Always;

    options.ExpireTimeSpan = TimeSpan.FromDays(7);

    options.SlidingExpiration = true;

    // Това е API: отговаря се със статус код вместо с пренасочване към страница за вход.
    options.Events.OnRedirectToLogin = context =>
    {
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;

        return Task.CompletedTask;
    };
});
```

Приложение 8: Форма за обява — пълна замяна на записа (`jobForm.ts`)

Заявката `PUT /api/jobs/{id}` заменя цялата обява, а не само подадените полета. Затова формата описва **ВСИЧКИ** полета, които крайната точка записва, и се попълва от една функция — иначе пропуснатото поле не се запазва, а се изтрива.

```
export type JobFormState = {  
  
  title: string  
  
  company: string  
  
  companyDescription: string  
  
  location: string  
  
  salary: string  
  
  jobType: string  
  
  workMode: string  
  
  employmentType: string  
  
  seniorityLevel: string  
  
  description: string  
  
  requirements: string  
  
  responsibilities: string  
  
  benefits: string  
  
  /** `YYYY-MM-DD`, или празно за липсващ краен срок. */  
  
  deadline: string  
  
  tags: string[]  
  
}
```

```
/** Изгражда състоянието на формата от обява, върната от API-то. */
```

```
export function jobFormFrom(job: Partial<JobPosting>): JobFormState {
```

```
  return {
```

```
    title: job.title ?? "",
```

```
    company: job.company ?? "",
```

```
    companyDescription: job.companyDescription ?? "",
```

```
    location: job.location ?? "",
```

```
    salary: job.salary || '$ 0',
```

```
    jobType: job.jobType || 'Full-time',
```

```
    workMode: job.workMode ?? "",
```

```
    employmentType: job.employmentType ?? "",
```

```
    seniorityLevel: job.seniorityLevel ?? "",
```

```
    description: job.description ?? "",
```

```
    requirements: job.requirements ?? "",
```

```
    responsibilities: job.responsibilities ?? "",
```

```
    benefits: job.benefits ?? "",
```

```
    deadline: toDateInput(job.deadline),
```

```
    tags: job.tags ?? [],
```

```
  }
```

```
}
```

Приложение 9: Структура на проекта

findajob/

- | — backend/
 - | | — Controllers/ Auth, Jobs, Application, Profiles, Cv, Messages,
Friendships, Notifications, SavedJobs, Admin, Media
 - | | — Data/ DbContext и добавящият seeder
 - | | — Models/ Обекти на базата и споделени речници за роли/статуси
 - | | — Services/ JobService, FileStorage, EmailService, SearchPattern
 - | | — Migrations/ Миграции на EF Core (7 броя)
 - | | — EmailTemplates/ HTML шаблони за писмата
 - | | — SeedAssets/ Фирмени логa, копирани в хранилището при първо стартиране
 - | | — App_Data/uploads/ Качени от потребителите файлове (извън системата за контрол на версиите)
- | — backend.tests/
 - | | — ApiFactory.cs Тестов хост за интеграционните тестове
 - | | — AuthorizationTests.cs Проверки на правата
 - | | — PasswordAndOwnershipTests.cs Пароли и собственост върху обявите
 - | | — JobServiceTests.cs Модулни тестове на услугата за обяви
 - | | — DataIntegrityTests.cs Гаранции на базата данни
 - | | — FileStorageTests.cs Хранилище за файлове
 - | | — DomainConstantsTests.cs Споделени речници

- | — frontend/src/
 - | | — components/ Обвивка, защита по роля, диалози, контроли за списъци
 - | | | — providers/ Контексти: автентикация, известия, съобщения, потвърждения
 - | | — pages/ Компоненти-страници, групирани по роля
 - | | — types.ts Типове на отговорите от API-то
 - | | — constants.ts Споделени списъци и правила за паролата
 - | | — jobForm.ts Състояние на формата за обява
 - | | — usePagedList.ts Hook за странирани списъци
 - | | — api.ts Axios клиент и нормализация на грешките
- | — Dockerfile Многоетапен образ
- | — fly.toml Конфигурация за разгръщане
- | — README.md Техническа документация за разработчици